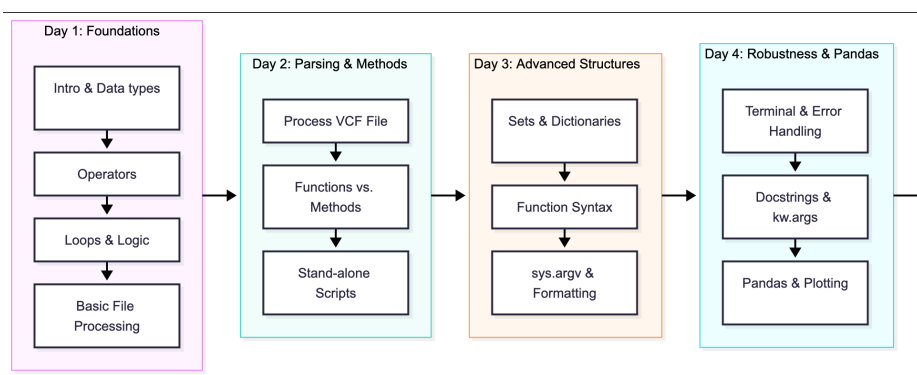
Introduction to



with Application to Bioinformatics

- Day 2

Yesterday's Key Takeaways



- Literals, variables
    - Literal: `3.14`, `"fruites"`
    - Variable: `dna`, `myVar`
    - Basic data types: `Numeric`, `String`, `Boolean`, `Collection`
- Built-in functions and operations
    - `print`, `max`, `sum`, `+`
- Loops
    - `for` loop and `while` loop
- Use `if/elif/else` statements to control the logic of the
- Read and write data from/to file
    - `open(filename, "r")` or `open(filename)`
    - `open(filename, "w")`

Day 2

- **Session 1**
    - Quiz: Review of Day 1
    - Lecture: Go through the quiz and review the key c
- **Session 2**
    - Lecture: Variants identification with VCF
    - Ex1:
    - PyQuiz 2.1

- **LUNCH**

- **Session 3**
    - Lecture: Functions and Methods
    - Ex2: continuation of Ex1
- **Session 4**
    - Standalone Python script
    - PyQuiz 2.2
    - Ex3: IMDb
- **Project time**

Quiz: Review Day 1

Go to Canvas, `Modules -> Day 2 -> Review Day 1`

Questions can be multiple-choice

~20 minutes

Q1 & Q2: Variables and Types

Q1. Which of the following is of the type `float`?:

```
7.4
```

Q2. Match the following variables with their type:

```
var1 = 54                    # integer
var2 = [1,2,7,1,24]          # list
var3 = 2.98                  # float
var4 = True                  # boolean
```

Literals and variables

- A literal is the **actual value itself**
- A variable is a **named container** that holds or points to a va

In [ ]:
```
"ATGCGTAC" # string
8 # integer
3.14 # float
True # boolean
```

In [ ]:
```
s = "ATGCGTAC"
a = 8
b = 3.14
c = True
```

Literals and variables

- A literal is the **actual value itself**
- A variable is a **named container** that holds or points to a va

In [ ]:
```
"ATGCGTAC" # string
8 # integer
3.14 # float
True # boolean
```

In [ ]:
```
s = "ATGCGTAC"
a = 8
b = 3.14
c = True
```

In [ ]:
```
# Use meaningful variable names !!
dna_sequence = "ATGCGTAC"
sequence_length = len(dna_sequence)
print("length of", dna_sequence, "is", sequence_le
```

Lists

A collection of values.

In [ ]:
```
x = [1,5,3,7,8]
y = ['a','b','c']
print(type(y))
# print(x + y) # a list can have heterogenous data
```

Q3: Comments

Which of the following symbols can be used to write comments

```
#
```

Q3: Comments

Which of the following symbols can be used to write comments

```
#
```

You can also use docstring `"""` to comment out multiple lines of
designed for that.

In [ ]:
```
"""
This is a description
of the code below
"""
li = ["abc"] * 10
print(li)
```

Q4 - Q6: Operations

Q4. What happens if you do `[1,2,5,11] + [87,2,43,3]`?

`[1,2,5,11,87,2,43,3]` The lists will be concatenated

Q5. How do you find out if the variable `x` is present in a the lis

Two correct answers:

1.

```
x in mylist
```

2.

```
for l in mylist:
    if l == x:
        print('Found a match')
```

Q6. How do you find out if 5 is larger than 3 and the integer 4 is
Fill in all the missing code.

```
5 > 3 and 4 == 4.0
```

Q6. How do you find out if 5 is larger than 3 and the integer 4 is
Fill in all the missing code.

```
5 > 3 and 4 == 4.0
```

```
5 > 3 and 4 == float(4)
```

Q6. How do you find out if 5 is larger than 3 and the integer 4 is
Fill in all the missing code.

```
5 > 3 and 4 == 4.0
```

```
5 > 3 and 4 == float(4)
```

In [ ]:
```
a = 0.1+0.2
b = 0.3
print(a == b)
```

Q6. How do you find out if 5 is larger than 3 and the integer 4 is
Fill in all the missing code.

```
5 > 3 and 4 == 4.0
```

```
5 > 3 and 4 == float(4)
```

In [ ]:
```
a = 0.1+0.2
b = 0.3
print(a == b)
```

In [ ]:
```
# Unsafe:
print(a == b)  # False

# Safe:
import math
print(math.isclose(a, b))  # True
```

Basic operations

Type	Operations
int	+ - * / ** % // ...
float	+ - * / ** % // ...
string	+ *

In [ ]:
```
i = 2
f = 5.46
li1 = [1,2,3,4]
li2 = [5,6,7,8]
```

In [ ]:
```
i * f
```

In [ ]:
```
li1 * i
```

Basic operations

Type	Operations
int	+ - * / ** % // ...
float	+ - * / ** % // ...
string	+ *

In [ ]:
```
i = 2
f = 5.46
li1 = [1,2,3,4]
li2 = [5,6,7,8]
```

In [ ]:
```
i * f
```

In [ ]:
```
li1 * i
```

In [ ]:
```
li1 * f # does this operation work?
```

Comparison/Logical/Membership operat

Operation	Meaning
<	less than
<=	less than or equal
>	greater than
>=	greater than or equal
==	equal
!=	not equal

Operation	Meaning
and	connects two st conditions havin
or	connects two st conditions havin
not	reverses and/or

Operation	Meaning
in	value in object
not in	value not in ob

In [ ]:
```
li = [1,2,3,4,5,6,7,8]
b = 5
b in li
```

In [ ]:
```
b not in li
# not b in li # works as well, but not recommended
```

In [ ]:
```
c = 10
b < c or c == 1
```

Python Operator Precedence (Highest →

Priority	Operator Category	Operators
1	Parentheses	`( )`
2	Exponentiation	`**`
3	Unary operators	`+x`, `-x`, `~x`
4	Multiplicative	`*`, `/`, `//`, `%`
5	Additive	`+`, `-`
6	Bitwise shifts	`<<`, `>>`
7	Bitwise AND	`&`
8	Bitwise XOR	`^`
9	Bitwise OR	`|`
10	Comparisons	`==`, `!=`, `<`, `<=`, `>`, `>=`, `is`, `is not`, `in`, `not in`
11	Boolean NOT	`not`
12	Boolean AND	`and`
13	Boolean OR	`or`

For beginners, use paretheses () if you are unsure

More details: https://www.w3schools.com/python/python_operator

Q7-Q9: Sequences and collecti

Q7. How do you select the second element in the variable `myli`

Q7-Q9: Sequences and collecti

Q7. How do you select the second element in the variable `myli`

Lists (and strings) are an ORDERED collection of elements where e
access through an index.

```
value:            4    3    8    10
index:            0    1    2    3
index(negative) -4   -3   -2   -1
```

In [ ]:
```
mylist = [4,3,8,10]
mylist[1]
```

Q8: Mutable / Immutable sequ
iterables

Lists are mutable object, meaning you can use an index to change
immutable and therefore not changeable.

In [ ]:
```
li1 = ['a','b','c']        # mutable
str1 = 'abc'               # immutable
```

In [ ]:
```
li1[2] = 'B'
li1
```

In [ ]:
```
str1[2] = 'B'
```

Q9: Which of the following ty
iterable?

An iterable sequence is anything you can loop over, ie, lists and st

- **lists**
- ingegers
- float
- **strings**
- booleans

New data type: `tuples`

- A tuple is almost like a list but an immutable sequence of ob
- Elements can NOT be changed in a tuple by using its index
- Still iterable

In [ ]:
```
myTuple = (1,2,3,4,'a','b','c')
myList = [1,2,3,4,'a','b','c']
```

In [ ]:
```
for item in myTuple:
    print(item)
```

In [ ]:
```
for item in myList:
    print(item)
```

In [ ]:
```
myList[2] = 42
myList
```

In [ ]:
```
myTuple[2] = 42
```

What is the benefit of `tuple`

- Immutability ensures data integrity and safety, in the use ca
  kept unchanged during processing
- Use less memory than `list`, better performance

Q10: If/ Else statements

How do you do to print 'Yes' if x is bigger than y?

```
if x > y:
    print('Yes')
```

Q10: If/ Else statements

How do you do to print 'Yes' if x is bigger than y?

```
if x > y:
    print('Yes')
```

In [ ]:
```
# elif
```

Q11-Q12: Files and loops

Q11: How do you open a file handle to read a file called 'somera

```
fh = open('somerandomfile.txt')
```

Q12: The file in the previous question contains several lines, ho line, with or without trailing whitespace?

1.

```
for line in fh:
    print(line)
```

2.

```
for row in fh:
    print(row.strip())
```

In [ ]:

```
fh = open('../files/somerandomfile.txt','r', encod
for line in fh:
    print(line.strip())
fh.close()
```

In [ ]:

```python
# while loop
key = ""
while key != "python2025":
    key = input("Enter your key: ")

print("Correct key!")
```

Questions?

Break (10 min)

Session 2

- Lecture: **Variants identification with VCF**
- Ex1
- PyQuiz 2.1

Lunch

Variants identification with V

- A VCF (Variant Call Format) file is a text file format used in b
  gene sequence variations.

Description of the task

You have a VCF file (`genotypes_small.vcf`, which can be fou
folder) with a number of samples. You are interested in only one of
and one region (**chr5, 1,200,000-1,300,000**, including the start a
you want to know is whether this sample has any variants in this re
variants.

What is your input?

```
##fileformat=VCFv4.2
##source=ExampleVCF
##FORMAT=<ID=GT,Number=1,Type=String,Description="Genotype">
##FORMAT=<ID=DP,Number=1,Type=Integer,Description="Read Depth">
##FORMAT=<ID=CB,Number=1,Type=String,Description="Called by S(Sanger), M(UMich),
#CHROM  POS ID  REF ALT QUAL    FILTER  INFO    FORMAT  Sample1 Sample2 Sample3
1   10177   rs367896725 A   AC  50  PASS    .   GT:DP:CB    0/1:30:SM   0/0:28:S
1   13110   rs569393146 G   A,C,T   99  PASS    .   GT:DP:CB    0/2:60:SM   1/1
```

1. **CHROM**: 1 (Chromosome 1)
2. **POS**: 10177 (Position 10177 on the chromosome, **starting f**
3. **ID**: rs367896725 (Reference SNP ID)
4. **REF**: A (Reference allele is A)
5. **ALT**: AC (Alternative allele is AC)
6. **QUAL**: 50 (Quality score)
7. **FILTER**: PASS (status, meaning passed all quality control)
8. **FORMAT**: GT:DP:CB (Genotype, Depth, Cell Barcode)
9. **SAMPLES**: 0/1:30:SM (Sample genotypes and additional in

Genotype field

```
##fileformat=VCFv4.2
##source=ExampleVCF
##FORMAT=<ID=GT,Number=1,Type=String,Description="Genotype">
##FORMAT=<ID=DP,Number=1,Type=Integer,Description="Read Depth">
##FORMAT=<ID=CB,Number=1,Type=String,Description="Called by S(Sanger), M(UMich),
#CHROM  POS ID  REF ALT QUAL    FILTER  INFO    FORMAT  Sample1 Sample2 Sample3
1   10177   rs367896725 A   AC  50  PASS    .   GT:DP:CB    0/1:30:SM   0/0:28:S
1   13110   rs569393146 G   A,C,T   99  PASS    .   GT:DP:CB    0/2:60:SM   1/1
```

`0/1`

- For human, and many other organisms, there are two copie
  inherited from both parents.
- `0` means DNA at this position is the same as reference alle
- `1` means has alternative allele, in this case `AC`

Genotype field

```
##fileformat=VCFv4.2
##source=ExampleVCF
##FORMAT=<ID=GT,Number=1,Type=String,Description="Genotype">
##FORMAT=<ID=DP,Number=1,Type=Integer,Description="Read Depth">
##FORMAT=<ID=CB,Number=1,Type=String,Description="Called by S(Sanger), M(UMich),
#CHROM  POS ID  REF ALT QUAL    FILTER  INFO    FORMAT  Sample1 Sample2 Sample3
1   10177   rs367896725 A   AC  50  PASS    .   GT:DP:CB    0/1:30:SM   0/0:28:S
1   13110   rs569393146 G   A,C,T   99  PASS    .   GT:DP:CB    0/2:60:SM   1/1
```

`0/1`

- For human, and many other organisms, there are two copie
  inherited from both parents.
- `0` means DNA at this position is the same as reference alle
- `1` means has alternative allele, in this case `AC`

The notation of alternative allele is not always `1`: `0/2`

- `2` means it is the second alternative allele, i.e. `C` in this ca

Some terminologies

Genotype	Status	Me
0/0	Homozygous Reference (wild type)	Tw ref
0/1 or **1/0**	Heterozygous	On ref on alt
1/1	Homozygous Alternate	Tw alt

Start with pseudocode!

- Pseudocode is a description of what you want to do withou
  syntax

Task:

You have a VCF file with a number of samples. You are interested i
samples (sample1) and one region (chr5, 1,200,000-1,300,000, inc
positions). What you want to know is whether this sample has any
and if so, which variants.

```
##fileformat=VCFv4.2
##source=ExampleVCF
##FORMAT=<ID=GT,Number=1,Type=String,Description="Genotype">
##FORMAT=<ID=DP,Number=1,Type=Integer,Description="Read Depth">
##FORMAT=<ID=CB,Number=1,Type=String,Description="Called by S(Sanger), M(UMich),
#CHROM  POS ID  REF ALT QUAL    FILTER  INFO    FORMAT  Sample1 Sample2 Sample3
1   10177   rs367896725 A   AC  50  PASS    .   GT:DP:CB    0/1:30:SM   0/0:28:S
1   13110   rs569393146 G   A,C,T   99  PASS    .   GT:DP:CB    0/2:60:SM   1/1
```

- Open the file and loop over lines (ignore lines with started w
- Identify lines where chromosome is equal to `5` and position
  and 1,300,000
- Isolate the column that contains the genotype for sample1,
- Extract only the genotypes (e.g. `0/1`) from the column
- Check if the genotype contains any alternate alleles, i.e., if n
- Print any variants containing alternate alleles for this sample
  region

Open the file and loop over lines (ignore lines starting with `#`), record

```
##fileformat=VCFv4.2
##source=ExampleVCF
##FORMAT=<ID=GT,Number=1,Type=String,Description="Genotype">
##FORMAT=<ID=DP,Number=1,Type=Integer,Description="Read Depth">
##FORMAT=<ID=CB,Number=1,Type=String,Description="Called by S(Sanger), M(UMich),
#CHROM  POS ID  REF ALT QUAL    FILTER  INFO    FORMAT  Sample1 Sample2 Sample3
1   10177   rs367896725 A   AC  50  PASS    .   GT:DP:CB    0/1:30:SM   0/0:28:S
1   13110   rs569393146 G   A,C,T   99  PASS    .   GT:DP:CB    0/2:60:SM   1/1
```

In [ ]:

```
with open('../downloads/genotypes_small.vcf', 'r',
    for line in fh:
        if not line.startswith('#'):
            line = line.strip()
            print(line)
            break
# Next, find chromosome 5
```

In [ ]:

Identify lines where chromosome is `5`

```
##fileformat=VCFv4.2
##source=ExampleVCF
##FORMAT=<ID=GT,Number=1,Type=String,Description="Genotype">
##FORMAT=<ID=DP,Number=1,Type=Integer,Description="Read Depth">
##FORMAT=<ID=CB,Number=1,Type=String,Description="Called by S(Sanger), M(UMich),
#CHROM  POS ID  REF ALT QUAL    FILTER  INFO    FORMAT  Sample1 Sample2 Sample3
1   10177   rs367896725 A   AC  50  PASS    .   GT:DP:CB    0/1:30:SM   0/0:28:S
1   13110   rs569393146 G   A,C,T   99  PASS    .   GT:DP:CB    0/2:60:SM   1/1
```

In [ ]:

```
with open('../downloads/genotypes_small.vcf', 'r',
    for line in fh:
        if not line.startswith('#'):
            line = line.strip()
            cols = line.split('\t')
            chrom = cols[0]
            if chrom == '5':
                print(cols)
                break

# Next, find the correct region, between 1,200,000
```

Identify lines where chromosome is `5` and position is between `1,300,000`

```
##fileformat=VCFv4.2
##source=ExampleVCF
##FORMAT=<ID=GT,Number=1,Type=String,Description="Genotype">
##FORMAT=<ID=DP,Number=1,Type=Integer,Description="Read Depth">
##FORMAT=<ID=CB,Number=1,Type=String,Description="Called by S(Sanger), M(UMich),
#CHROM  POS ID  REF ALT QUAL    FILTER  INFO    FORMAT  Sample1 Sample2 Sample3
1   10177   rs367896725 A   AC  50  PASS    .   GT:DP:CB    0/1:30:SM   0/0:28:S
1   13110   rs569393146 G   A,C,T   99  PASS    .   GT:DP:CB    0/2:60:SM   1/1
```

In [ ]:

```
with open('../downloads/genotypes_small.vcf', 'r',
    for line in fh:
        if not line.startswith('#'):
            line = line.strip()
            cols = line.split('\t')
            chrom = cols[0]
            pos = int(cols[1])
            if chrom == '5' and (1200000 <= pos <=
                print(cols)
                break
# Next, find the genotypes for sample1
```

Isolate the column that contains the genotype for sample1

```
##fileformat=VCFv4.2
##source=ExampleVCF
##FORMAT=<ID=GT,Number=1,Type=String,Description="Genotype">
##FORMAT=<ID=DP,Number=1,Type=Integer,Description="Read Depth">
##FORMAT=<ID=CB,Number=1,Type=String,Description="Called by S(Sanger), M(UMich),
#CHROM  POS ID  REF ALT QUAL    FILTER  INFO    FORMAT  Sample1 Sample2 Sample3
1   10177   rs367896725 A   AC  50  PASS    .   GT:DP:CB    0/1:30:SM   0/0:28:S
1   13110   rs569393146 G   A,C,T   99  PASS    .   GT:DP:CB    0/2:60:SM   1/1
```

In [ ]:

```
with open('../downloads/genotypes_small.vcf', 'r',
    for line in fh:
        if not line.startswith('#'):
            line = line.strip()
            cols = line.split('\t')
            chrom = cols[0]
            pos = int(cols[1])
            if chrom == '5' and (1200000 <= pos <=
                gt_field = cols[9]
                print(gt_field)
                break

# Next, extract the genotypes only
```

Extract only the genotypes from the column

```
##fileformat=VCFv4.2
##source=ExampleVCF
##FORMAT=<ID=GT,Number=1,Type=String,Description="Genotype">
##FORMAT=<ID=DP,Number=1,Type=Integer,Description="Read Depth">
##FORMAT=<ID=CB,Number=1,Type=String,Description="Called by S(Sanger), M(UMich),
#CHROM  POS ID  REF ALT QUAL    FILTER  INFO    FORMAT  Sample1 Sample2 Sample3
1   10177   rs367896725 A   AC  50  PASS    .   GT:DP:CB    0/1:30:SM   0/0:28:S
1   13110   rs569393146 G   A,C,T   99  PASS    .   GT:DP:CB    0/2:60:SM   1/1
```

In [ ]:

```
with open('../downloads/genotypes_small.vcf', 'r',
    for line in fh:
        if not line.startswith('#'):
            line = line.strip()
            cols = line.split('\t')
            chrom = cols[0]
            pos = int(cols[1])
            if chrom == '5' and (1200000 <= pos <=
                gt_field = cols[9]
                gt = gt_field.split(':')[0]
                print(gt)
                break

# Next, find in which positions sample1 has altern
```

Check if the genotype contains any alternate alleles

```
##fileformat=VCFv4.2
##source=ExampleVCF
##FORMAT=<ID=GT,Number=1,Type=String,Description="Genotype">
##FORMAT=<ID=DP,Number=1,Type=Integer,Description="Read Depth">
##FORMAT=<ID=CB,Number=1,Type=String,Description="Called by S(Sanger), M(UMich),
#CHROM  POS ID  REF ALT QUAL    FILTER  INFO    FORMAT  Sample1 Sample2 Sample3
1   10177   rs367896725 A   AC  50  PASS    .   GT:DP:CB    0/1:30:SM   0/0:28:S
1   13110   rs569393146 G   A,C,T   99  PASS    .   GT:DP:CB    0/2:60:SM   1/1
```

In [ ]:

```
with open('../downloads/genotypes_small.vcf', 'r',
    for line in fh:
        if not line.startswith('#'):
            line = line.strip()
            cols = line.split('\t')
            chrom = cols[0]
            pos = int(cols[1])
            if chrom == '5' and (1200000 <= pos <=
                gt_field = cols[9]
                gt = gt_field.split(':')[0]
                if gt != "0/0": # For robust VCF p
                    print(gt)
#Next, print nicely
```

Print any variants containing alternate alleles for this sample be

```
##fileformat=VCFv4.2
##source=ExampleVCF
##FORMAT=<ID=GT,Number=1,Type=String,Description="Genotype">
##FORMAT=<ID=DP,Number=1,Type=Integer,Description="Read Depth">
##FORMAT=<ID=CB,Number=1,Type=String,Description="Called by S(Sanger), M(UMich),
#CHROM  POS ID  REF ALT QUAL    FILTER  INFO    FORMAT  Sample1 Sample2 Sample3
1   10177   rs367896725 A   AC  50  PASS    .   GT:DP:CB    0/1:30:SM   0/0:28:S
1   13110   rs569393146 G   A,C,T   99  PASS    .   GT:DP:CB    0/2:60:SM   1/1
```

In [ ]:

```
with open('../downloads/genotypes_small.vcf', 'r',
    for line in fh:
        if not line.startswith('#'):
            line = line.strip()
            cols = line.split('\t')
            chrom = cols[0]
            pos = int(cols[1])
            if chrom == '5' and (1200000 <= pos <=
                gt_field = cols[9]
                gt = gt_field.split(':')[0]
                if gt != "0/0": # For robust VCF p
                    ref = cols[3]
                    alt = cols[4]
                    info = chrom + ":" + str(pos)
                    print(info)
```

Exercise 1

Go to Canvas -> Moudles -> Exercises Day 2

Three options

1. Green exercise
2. Yellow exercise
3. Red exercise

The level of complexity increases with each exercise

- ChatGPT exercise

New to programming: Do Green exercise and possibly Yellow exer
More experienced: Do Yellow exercise and/or Red exercise + Chat

The exercise will be continued in the afte

Session 3

- Lecture: **Functions and Methods**
- Ex2:

What is a function

- A function is a block of code that performs a specific task.
- It can take input (parameters) and return output (results).
- Functions help to reuse code and make programs more mo
- With a function, you only need to know how to use it, but no

In [ ]:
```
print("this is your sequence")
```

In [ ]:
```
sorted([5,3,135,31])
```

In [ ]:
```
sum([1,2,3,4,5])
```

What is a method

- A method is a function that is associated with objects (insta
  - All data types in Python are objects

In [ ]:
```
"   I'm a line with trailing spaces  ".strip()
```

In [ ]:
```
"   I'm a line with trailing spaces  ".split()
```

In [ ]:
```
li = [3, 1, 5, 43, 5]
li.sort()
print(li)
```

What does it matter to me?

For now, you only need to know the different syntaxes of using a fu
and how to use them

A function:
`functionName(arg1, arg2)`

A method:
`obj.methodName()`

Python built-in functions:

https://docs.python.org/3/library/functions.html

Built-in Functions			
abs( )	delattr( )	hash( )	memoryview( )
all( )	dict( )	help( )	min( )
any( )	dir( )	hex( )	next( )
ascii( )	divmod( )	id( )	object( )
bin( )	enumerate( )	input( )	oct( )
bool( )	eval( )	int( )	open( )
breakpoint( )	exec( )	isinstance( )	ord( )
bytearray( )	filter( )	issubclass( )	pow( )
bytes( )	float( )	iter( )	print( )
callable( )	format( )	len( )	property( )
chr( )	frozenset( )	list( )	range( )
classmethod( )	getattr( )	locals( )	repr( )
compile( )	globals( )	map( )	reversed( )
complex( )	hasattr( )	max( )	round( )

Introduction to some useful functions

help

In [ ]:
```
help(max)
```

In [ ]:
```
a = 5
help([1,2,3])
```

range

In [ ]:
```
for i in range(5):
    print(i)
```

range

In [ ]:
```
for i in range(5):
    print(i)
```

In [ ]:
```
# range(start, stop, step)
# generate a series of odd number
for i in range(1, 10, 2):
    print(i)
```

range

In [ ]:
```
for i in range(5):
    print(i)
```

In [ ]:
```
# range(start, stop, step)
# generate a series of odd number
for i in range(1, 10, 2):
    print(i)
```

In [ ]:
```
# negative step
for i in range(10, 1, -2):
    print(i)
```

range

In [ ]:
```
for i in range(5):
    print(i)
```

In [ ]:
```
# range(start, stop, step)
# generate a series of odd number
for i in range(1, 10, 2):
    print(i)
```

In [ ]:
```
# negative step
for i in range(10, 1, -2):
    print(i)
```

Strictly speaking, range is not a function, instead, it is a built-in

In [ ]:
```
print(type(range))
print(type(sum))
```

sorted

In [ ]:
```
li = [5, 4, 1, 3, 10, 13, 13]
print(sorted(li))
```

sorted

In [ ]:
```
li = [5, 4, 1, 3, 10, 13, 13]
print(sorted(li))
```

In [ ]:
```
print(sorted(li, reverse=True))
```

sorted

In [ ]:
```
li = [5, 4, 1, 3, 10, 13, 13]
print(sorted(li))
```

In [ ]:
```
print(sorted(li, reverse=True))
```

In [ ]:
```
# sorted does not change the content in place
li = [5, 4, 1, 3, 10, 13, 13]
print(sorted(li))
print(li)
sorted_li = sorted(li)
print(li)
```

`dir`:

return a list of names comprising the attributes of the given object

In [ ]:
```
dir(str)
```

In [ ]:
```
a = "This is an ongoing course".strip().split().in
dir(a)
```

Some userful methods for Strings

String Methods	
str.strip( )	str.startswith( )
str.rstrip( )	str.endswith( )
str.lstrip( )	str.upper( )
str.split( )	str.lower( )
str.join( )	

str.split()

In [ ]:
```
a = '  split a "string into a list" '
a.split()
```

str.split()

In [ ]:
```
a = '  split a "string into a list" '
a.split()
```

In [ ]:
```
a.split(maxsplit=2)
```

str.lstrip()

str.**lstrip**([*chars*])

Return a copy of the string with leading characters removed. The *chars* argument is a s
set of characters to be removed. If omitted or `None`, the *chars* argument defaults to rem
The *chars* argument is not a prefix; rather, all combinations of its values are stripped:

```
>>> '   spacious   '.lstrip()
'spacious   '
>>> 'www.example.com'.lstrip('cmowz.')
'example.com'
```

In [ ]:
```
line = "TAG:here comes the actual content"
line.lstrip("TAG:")
print(line.lstrip("TAG"))
print(line) # the content of line is not changed i
```

In [ ]:
```
line = "AB:A:B should not be removed"
line.lstrip("AB:") # The chars argument is not a p
```

str.lstrip()

str.**lstrip**([*chars*])

Return a copy of the string with leading characters removed. The *chars* argument is a s
set of characters to be removed. If omitted or `None`, the *chars* argument defaults to rem
The *chars* argument is not a prefix; rather, all combinations of its values are stripped:

```
>>> '   spacious   '.lstrip()
'spacious   '
>>> 'www.example.com'.lstrip('cmowz.')
'example.com'
```

In [ ]:
```
line = "TAG:here comes the actual content"
line.lstrip("TAG:")
print(line.lstrip("TAG"))
print(line) # the content of line is not changed i
```

In [ ]:
```
line = "AB:A:B should not be removed"
line.lstrip("AB:") # The chars argument is not a p
```

In [ ]:
```
line.removeprefix("AB:")
```

str.join

In [ ]:
```
','.join(["ATGC", "CCCCC", "AAATG", "CTTTGTA"]) #
```

Some userful methods for Lists

Operation	Result
s.append(x)	appends *x* to the end of the sequence
s.insert(i, x)	*x* is inserted at pos *i*
s.pop([i])	retrieves the item *i* from *s* and also removes it
s.remove(x)	retrieves the first item from *s* where s[i] == x
s.reverse( )	reverses the items of *s* in place

In [ ]:
```
a = [1, 2, 3, 4, 5, 5, 5, 5]
a.append(6)
a
```

In [ ]:
```
a.pop()
a
```

In [ ]:
```
a.pop(1) # pop(index)
a
```

In [ ]:
```
a.reverse()
a
```

In [ ]:
```
a.remove(5) # remove(value)
a
```

In [ ]:
```
a.insert(0, 13)
a
```

In [ ]:
```
### Tuples are immutable
a = (2, 3, 4, 15)
a.pop()
```

Summary

- Tuples are immutable sequences of objects
- Plan your approach before you start coding
- A method always belongs to an object of a specific class, a
  to
    - call a function by `functionName(args)`
    - call a method by `obj.methodName(args)`
- The official Python documentation describes the syntax for
  methods
    - https://docs.python.org/3/
    - https://www.w3schools.com/python/

Exercise 2 (continuation of Ex1

Three options

1. Green exercise
2. Yellow exercise
3. Red exercise

The level of complexity increases with each exercise.

- ChatGPT exercise

New to programming: Do Green exercise and possibly Yellow exer
More experienced: Do Yellow exercise and/or Red exercise + Chat

Session 4

- Lecture: **Standalone python script**
- PyQuiz 2.2!
- Ex3

Project time

Python script

A Python script is a file containing Python code that can be execut

- File Extension: `.py`

In [ ]:

```
with open('../downloads/genotypes_small.vcf', 'r',
    for line in fh:
        if not line.startswith('#'):
            line = line.strip()
            cols = line.split('\t')
            chrom = cols[0]
            pos = int(cols[1])
            if chrom == '5' and (1200000 <= pos <=
                gt_field = cols[9]
                gt = gt_field.split(':')[0]
                if gt != "0/0": # For robust VCF p
                    ref = cols[3]
                    alt = cols[4]
                    info = chrom + ":" + str(pos)
                    print(info)
```

Now we will save the code in a text file and try to run it in the te

About the file extension `.py`

- Actually a Python script can have any extention or even wit
- We use it as a convention
- Other benefits of using the `.py` entension - people and th

Things to remember when working with

- Put *#!/usr/bin/env python* in the beginning of the file
- Make the file executable to run with `./script.py`
- Otherwise run script with `python script.py`

5 mins exercise

Put some of your Python code you have written so far and save it i
in a terminal with

```
python yourscript.py
```

- Replace `yourscript.py` with the actual name you use

PyQuiz 2.2

Go to Canvas, `Modules -> Day 2 -> PyQuiz 2.2`

Exercise with IMDb

Go to Canvas, `Modules -> Day 2 -> IMDb exercise - d`

IMDb

Get the file `250.imdb` from the download folder (https://raw.githu
NBISweden/workshop-python/ht25/downloads.zip)

The format of this file is:

- Line by line
- Columns separated by the | character
- Header starting with #

```
# Votes | Rating | Year | Runtime | URL | Genres | Title
   126807|    8.5|1957|5280|https://images-na.ssl-images....|Drama,War|Paths of G
    71379|    8.2|1925|4320|https://images-na.ssl-images....|Adventure,Comedy,Dra
```

Votes | Rating | Year | Runtime | URL | Genres | Title

Task: Find the movie with the highest rat

# Votes	Rating	Year	Runtime	URL	Genres	Title
126807	8.5	1957	5280	https://images-na.ssl-images....	Drama,War	Paths of G
71379	8.2	1925	4320	https://images-na.ssl-images....	Adventure,Comedy,Dra	

Write step-by-step pseudocode

# Votes	Rating	Year	Runtime	URL	Genres	Title
126807	8.5	1957	5280	https://images-na.ssl-images....	Drama,War	Paths of G
71379	8.2	1925	4320	https://images-na.ssl-images....	Adventure,Comedy,Dra	

- Open file
- Initiate counter to keep track of highest rating and movie. S
- Loop over all lines not starting with '#'
- Strip and split the lines into a list
- Save the element containing the rating from the list into a v
- If current rating is higher than the rating in the counter, repl rating and movie
- Close file
- Print counter

```
# Votes | Rating | Year | Runtime | URL | Genres | Title
   126807|   8.5|1957|5280|https://images-na.ssl-images....|Drama,War|Paths of G
    71379|   8.2|1925|4320|https://images-na.ssl-images....|Adventure,Comedy,Dra
```

In [ ]:

```
fh   = open('../downloads/250.imdb', 'r', encoding
best = [0,'']            # here we save the rating
for line in fh:
    if not line.startswith('#'):
        cols   = line.strip().split('|')
        rating = float(cols[1].strip())
        if rating > best[0]:            # if the ra
            best = [rating,cols[6]]
fh.close()
print(best)
```